

Zadania praktyczne z programowania aplikacji Hono

Na spotkaniu przedstawiłem zupełne fundamenty programowania przy użyciu Express.js. Jest jednak od niego wiele lepszych opcji. Docelowo będziemy używać ElysiaJS (możliwe, że będzie o nim już na następnym spotkaniu), ale przedtem poznamy programowanie REST API bez lukru typowego.

Zarządziłem więc zmianę frameworka z Express.JS na HonoJS. Historycznie jestem związany tak, że najpierw korzystałem z Express.JS, później sporawy projekt przepisałem na HonoJS, a po dwóch tygodniach poznałem ElysiaJS... i ponownie przepisałem ten sam projekt. Z pozoru niezrozumiała decyzja, w efekcie okazała się bardzo dobra. Tak czy inaczej, HonoJS jest lepszą alternatywą dla Express.JS, ale nauczymy się tych samych konceptów.

Ta lista składa się z trzech zasadniczych części:

- setup projektu,
- poznanie Postmana,
- dodawanie funkcji.

Każda część zależy od poprawnego wykonania poprzednich.

Aby otrzymać punkty za pierwsze dwie części wystarczy porobić parę screenshotów w aplikacji Postman.

Przy ostatniej części należy robić commity co każde zadanie, by zapisać stan z wykonanym zadaniem (a nie wszystkich zadań na raz).

Treści zadań rozpoczynają się na kolejnej stronie

Obietnice (Promisy) wytłumaczone są na ostatniej stronie tej listy

Zadanie A. (4 pkt)

Setup projektu

1. (1 pkt)

Sklonuj [to repozytorium](#). Przejdź do folderu w4, skopiuj folder hono i przenieś do swojego foldera z rozwiązaniami. W folderze uruchom polecenie `bun install` aby zainstalować zależności.

2. (1 pkt)

Wejdź do kodu który zaczyna się w `index.ts`, przeczytaj go zwracając uwagę na dokumentację. W przypadku niezrozumienia czegoś, szukaj pomocy w [dokumentacji HonoJS](#)

3. (1 pkt)

Uruchom aplikację napisaną w HonoJS. Do tego celu służy jedno polecenie. Spróbuj sobie przypomnieć ze spotkania jak ono brzmiało. Przed poddaniem się i spojrzeniem do wskazówki sprawdź dokumentację Bun.

Sprawdź czy serwer nasłuchuje, przechodząc na <http://localhost:3000/>

Wskazówka

W jaki sposób uruchomić plik `.js/.ts`? Jak to robiliśmy przy użyciu `node`? Wystarczy zamienić `node` na `bun`.
Serwer sam zacznie nasłuchiwać na porcie 3000.

4. (1 pkt)

Uruchom aplikację w trybie `hot reloading`, czyli zmiana kodu implikuje dynamiczne przeładowanie aplikacji.

Sprawdź czy działa, przechodząc na <http://localhost:3000/>, zmieniając endpoint GET pod ścieżką `/`, aby zwrócił inną wiadomość i odświeżając stronę w przeglądarce.

Zadanie B. (2 pkt)

Konfiguracja Postmana do testowania REST API

1. (1 pkt)

[Dołącz do przestrzeni roboczej na Postmanie](#) (nie gwarantuję że link działa), przejdź do Collections > W4. Sprawdź jakie zapytania są zdefiniowane. Popatrz co jest podawane jako body, headers.

2. (1 pkt)

Otwórz zakładkę Flows, wybierz pierwszy Flow z oznaczeniem [0], uruchom na losowym Scenario. Zrób tak dla każdego flow z prefiksem [0]. Ten krok jest do sprawdzenia, czy aplikacja działa prawidłowo w Twoim środowisku.

Kontekst

Flow jest trochę taką linią produkcyjną po której jeżdżą dane, a maszynami na ich drodze są poszczególne endpointy. W REST API można wybrać ścieżki, które deterministycznie powinny za każdym razem dawać taki sam efekt.

Np. Dodaj użytkownika > (otrzymaliśmy jakieś id) > Usuń użytkownika (id) > Użytkownika nie ma.

Jeśli coś się zepsuje to łatwo można to zauważyć bez konieczności ręcznego weryfikowania. W zadaniu C. do każdego podzadania (ponumerowanego numerem podzadania) jest utworzony flow, który przy poprawnej realizacji zadania powinien się wykonać i dać poprawny wynik (brak error w obiekcie wynikowym). Kod nadal będzie podlegał ręcznej ocenie np. czytelności, ale może to dać wskazówkę czy kod robi mniej więcej to, co byśmy chcieli.

Zadanie C. (8 pkt)
Rozbudowa aplikacji REST API

Nadchodzi

Dodatek

Obietnice (Promisy)

Czym jest synchroniczność i asynchroniczność wykonywania programu? Okazuje się, że do tej pory cały nasz kod w JavaScriptcie (i zdecydowanej większości innych języków) był wykonywany synchronicznie, a więc sumiennie linijka po linijce. Wynikała z tego taka zależność, że

```
szybkaFunkcja()  
wolnaFunkcja()  
szybkaFunkcja2()
```

zajęłoby tyle czasu, ile potrzebują dwie szybkaFunkcja oraz jedna wolnaFunkcja. A ponadto, szybkaFunkcja2 zostałaby wykonana dopiero po wykonaniu wolnaFunkcja.

Wyobraźmy sobie że piszemy endpoint dla serwera, który pozwala na zresetowanie hasła. Ciało endpointu wyglądałoby pewnie jakoś tak:

```
({user, code}, res) => {  
  const code = generateCode();  
  smsService.sendCode(user.phoneNumber, code);  
  user.update({  
    ...user,  
    resetCode: code  
  })  
  user.saveToDatabase()  
  return res.json({  
    status: 200,  
    message: "Wysłano kod resetu hasła na numer telefonu połączony z kontem!"  
  })  
}
```

dla kontekstu: wysyłanie e-maili, a tym bardziej smsów zajmuje sporo czasu, serwisy wysyłające te sms-y są wiecznie zawalone, wolno odpowiadają na żądania wysłania smsów, a czasami w ogóle. metoda sendCode jest więc absurdalnie wolna, zajmuje powyżej 2 minut

Kod który faktycznie aktualizuje pole resetCode (który wprowadza użytkownik we frontendzie) jest uzależniony od wykonania sendCode. Uzależniony w złym tego słowa znaczeniu.

A jakie jest dobre znaczenie?

Założmy że serwis wysyłania SMSów nie działa, wtedy sms nigdy nie dojdzie do użytkownika, sendCode wyrzuci wyjątek który nie zostanie złapany, backend się wysypie, kod który nie musiał być ustawiony nie zostanie ustawiony w bazie danych, wszyscy szczęśliwi.

Jeżeli dodamy do frontendu funkcję samowprowadzania kodu z SMSów, to zakładamy że momentalnie po sendCode użytkownik wyśle kolejne żądanie, do innego endpointu z prośbą o resetowanie hasła i kodem który otrzymał. I serwer wyrzuci mu błąd, bo wysłał ten kod zbyt szybko, ponieważ linia odpowiedzialna za zmianę kodu się dopiero wykonuje.

Kolejnym błędem jest to, że return z odpowiedzią również zostanie przełożony na moment po zaktualizowaniu kodu w bazie. W rezultacie na frontendzie posypią się błędy, bo wiele zapytań nie otrzyma odpowiedzi w określonym czasie (zazwyczaj 30 sekund).

Rozwiązanie? - Konkurencja!

Skoro `sendCode` zajmuje tak dużo czasu, to powiedzmy JavaScriptowi "Ej, oblicz mi to, a ja sobie będę robił coś innego. Jak skończy to daj mi znać!". Czy to nie byłoby fajne?

Okazuje się, że tak właśnie działa asynchroniczność w JavaScriptcie. Możemy stworzyć nową funkcję korzystając tym razem z keyworda `async`:

```
async ({user, code}, res) => {  
}
```

Teraz dorzucamy te rzeczy, które wykonują się w normalny sposób. To są głównie jakieś obliczenia JavaScriptowe, odwołania do metod klas itd.

```
async ({user, code}, res) => {  
  const user = generateCode();  
  user.update({  
    ...user,  
    resetCode: code  
  })  
  
  return res.json({  
    status: 200,  
    message: "Wysłano kod resetu hasła na numer telefonu połączony z kontem!"  
  })  
}
```

No i zostały nam jeszcze wszelkie rzeczy które wymagają specjalnego traktowania. To są zazwyczaj funkcje:

- 1) ingerujące w sieć,
- 2) wykonujące zapytania na bazie danych,
- 3) ładujące pliki,
- 4) wykonujące inne funkcje asynchroniczne.

```
async ({user, code}, res) => {  
  const user = generateCode();  
  smsService.sendCode(user.phoneNumber, code);  
  user.update({  
    ...user,  
    resetCode: code  
  })  
  user.saveToDatabase()  
  
  return res.json({  
    status: 200,  
    message: "Wysłano kod resetu hasła na numer telefonu połączony z kontem!"  
  })  
}
```

Super! Osiągnęliśmy cel! Teraz funkcje które są asynchroniczne i wykonane w scope asynchronicznym zostaną odroczone, JS sobie z tym poradzi a my możemy lecieć dalej!

Ale coś jest nie tak... Co jeśli `sendCode` się nie powiedzie? Albo w `saveToDatabase` okaże się że baza danych wyleciała w powietrze? Co teraz?

Sprawdźmy jaki jest typ wynikowy metody `sendCode`. Pod warunkiem że to funkcja asynchroniczna, to musi być to coś takiego: `Promise<...>`. Te kropki oznaczają jaki typ nam zaoferuje funkcja po tym jak się wykona. `Promise` natomiast, to **obietnica** dana nam przez środowisko, że on ten kod sumiennie wykona. Ale jeśli chcemy wykorzystać to co wyleci z tej obietnicy, możemy ustawić jej

callback co ma się wykonać jak skończy swoje liczenie. Do tego celu korzystamy z metody `Promise<T>#then`:

```
smsService.sendCode(user.phoneNumber, code).then(response => {
  if(response.success){
    console.log("Super! Powiodło się")
  } else {
    console.warn("Nie udało się wysłać smsu...")
  }
})
```

Typem `response` jest tutaj ten trzykropek.

Jeżeli metoda `sendCode` może wyrzucić jakiś wyjątek, to możemy go złapać dodając callback metodą `Promise<T>#catch`:

```
smsService.sendCode(user.phoneNumber, code).then(response => {
  if(response.success){
    console.log("Super! Powiodło się")
  } else {
    console.warn("Nie udało się wysłać smsu...")
  }
}).catch(e => {
  console.log("O nie... System SMS znowu nie odpowiada!")
})
```

A gdybyśmy chcieli dodać jakiś kod który czyści po wykonanej obietnicy, bez względu na rezultat, możemy dodać callback metodą `Promise<T>#finally`:

```
user.lock.bind()
smsService.sendCode(user.phoneNumber, code).then(response => {
  if(response.success){
    console.log("Super! Powiodło się")
  } else {
    console.warn("Nie udało się wysłać smsu...")
  }
}).catch(e => {
  console.log("O nie... System SMS znowu nie odpowiada!")
}).finally(() => {
  // zwalnia locka, czyli blokadę nadpisu użytkownika przez wywołania w kodzie
  // w tym przykładzie to bezsensowny zabieg, nie pisze się takiego kodu w webie.
  // locki zazwyczaj stosują prawdziwe programy które zmieniają jakieś pliki lub
  // istnieje ryzyko że mogą działać na wielu wątkach jednocześnie gdy nie powinny.
  user.lock.free()
})
```

Jeżeli dodamy jakieś callbacki, to zazwyczaj chcielibyśmy, aby scope naszego wywołania nam nie uciekł. A to się stanie jeśli pozwolimy środowisku wykonać obietnicę w dowolnym momencie. Istnieje słowo kluczowe `await`, które wymaga na środowisku najpierw spełnienia danej obietnicy przed dalszym wykonywaniem. Zatem finalna wersja będzie wyglądać mniej więcej tak:

```

async ({user, code}, res) => {
  const user = generateCode();

  // zakładamy że ta metoda ingeruje w bazę danych, bo jednak lock zakładany jest
  zazwyczaj do uniknięcia konfliktów między procesami
  await user.lock.bind()
  await smsService.sendCode(user.phoneNumber, code).then(response => {
    if(response.success){
      console.log("Super! Powiodło się")
    } else {
      console.warn("Nie udało się wysłać smsu...")
    }
  }).catch(e => {
    console.log("O nie... System SMS znowu nie odpowiada!")
  }).finally(async () => {
    // zauważmy że jesteśmy w callbacku asynchronicznym. W kontekstach
    synchronicznych nie moglibyśmy korzystać z async
    await user.lock.free()
  })

  user.update({
    ...user,
    resetCode: code
  })
  // tutaj nie dodajemy callbacków przez then lub catch, a mimo to jest await, bo to
  Promise tak samo jak then wykonany na Promise
  await user.saveToDatabase()

  return res.json({
    status: 200,
    message: "Wysłano kod resetu hasła na numer telefonu połączony z kontem!"
  })
}

```

Inne kluczowe rzeczy to:

1) Konstruowanie obietnic

```

// konstruktor Promise to callback przyjmujący dwa parametry, będące callbackami.
Wywołanie resolve wypełnia obietnicę i zwraca wartość stanu "poprawnego",
natomiast reject wypełnia obietnicę i zwraca wartość stanu "niepoprawnego", błędu
const obietnica = new Promise((resolve, reject) => {
  setTimeout(() => resolve(42), 500)
})

obietnica.then(v => console.log(v))

```

2) Metody statyczne

Pozwalają zamienić wiele obietnic w jedną, której wykonanie zależy od wykonania wszystkich, niektórych, pierwszej obietnicy. [Dokumentacja stoi otworem](#)

3) Asynchrony poisoning

Wyobraźmy sobie ścieżkę wywołania 10 **synchronicznych** funkcji, gdzie funkcja A woła funkcję B, itd:

A -> B
B -> C
C -> D
D -> E
E -> F
F -> G
G -> H
H -> I
I -> J
J -> K

Funkcję K nazwiemy funkcją która leży najgłębiej w tej ścieżce. Jeżeli dopiszemy jedno słowo kluczowe do deklaracji funkcji K, a tym słowem kluczowym będzie `async`, to nagle cała logika do samej góry nam się sypie.

Wniosek z tego taki - korzystać z asynchroniczności **TAM GDZIE TO NIEZBĘDNE**, czyli między innymi wtedy gdy jakaś funkcja biblioteczna jest zdefiniowana jako asynchroniczna. Ktoś pewnie miał za tym dobry powód.